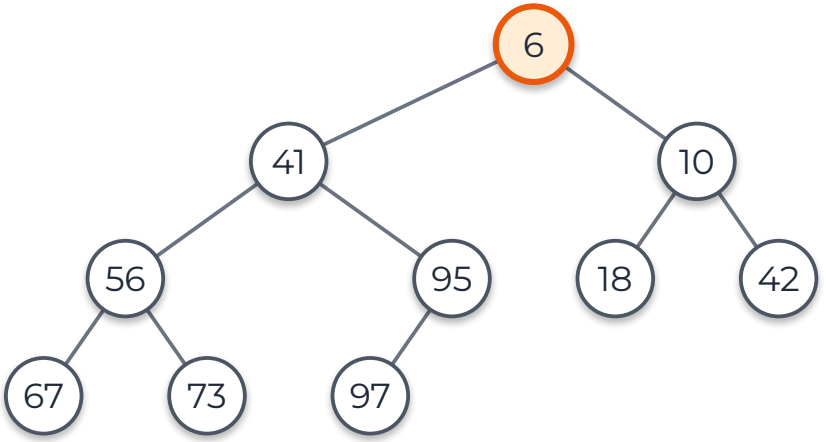
Heaps & Treaps

Binärbäume mit Heap-Eigenschaft

Heap: Definition

- **Min-Heap**: Knoten ≤ Kinder
- **Max-Heap**: Knoten ≥ Kinder
- Ab jetzt: **Heap = Min-Heap**
- **Kein** Suchbaum: links und rechts sind nicht sortiert
- Wurzel = kleinstes Element



(Min-)Heap: Eltern ≤ Kinder

Heap herstellen

Algorithmus	Vorbedingung an B	Einsatz
RotationHeap	höchstens **ein Blatt** ist falsch eingeordnet	Treaps
DownHeap („Sickern“)	alles in Ordnung, **außer** ggf. die Wurzel eines Teilbaums	Heap wiederherstellen
BuildHeap	keine – B ist ein **beliebiger** Binärbaum	Heap herstellen

DownHeap („Sickern“)

Situation

Nur die Wurzel n verletzt die Heap-Bedingung.

Ablauf

1. n mit seinen beiden Kindern vergleichen
2. n mit dem Kind tauschen, das den **kleineren** Schlüssel hat
3. beim neuen Platz von n weitermachen

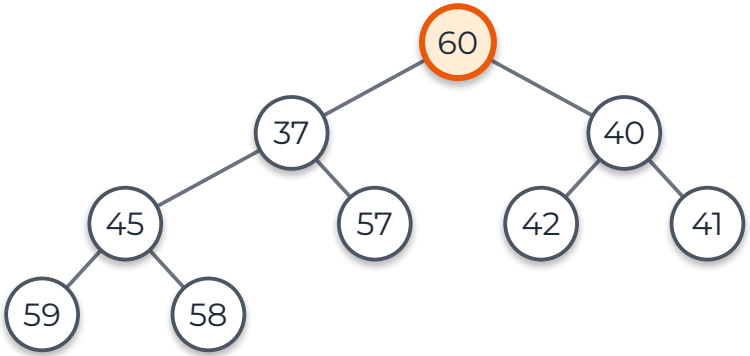
Ende: n erfüllt die Bedingung oder ist ein Blatt.

DownHeap(B, n)

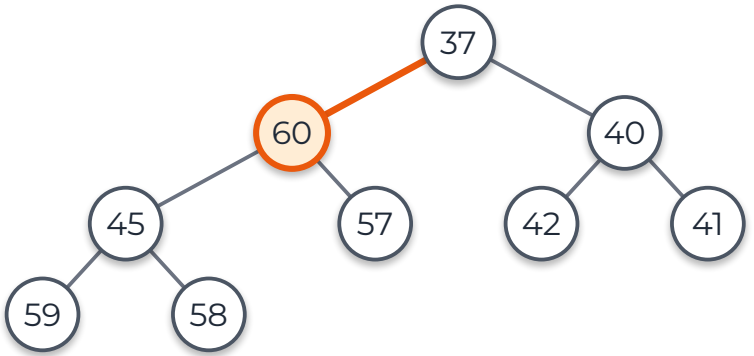
solange n die Bedingung verletzt:
    k ← Kind mit kleinerem Schlüssel
    tausche n und k
    n ← k

DownHeap: Beispiel

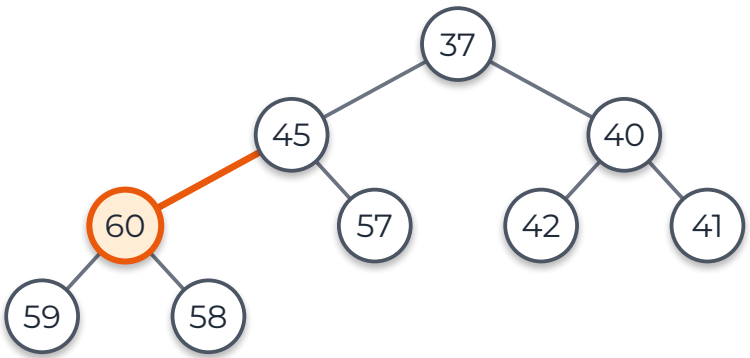
Start: nur die Wurzel 60 verletzt die Bedingung



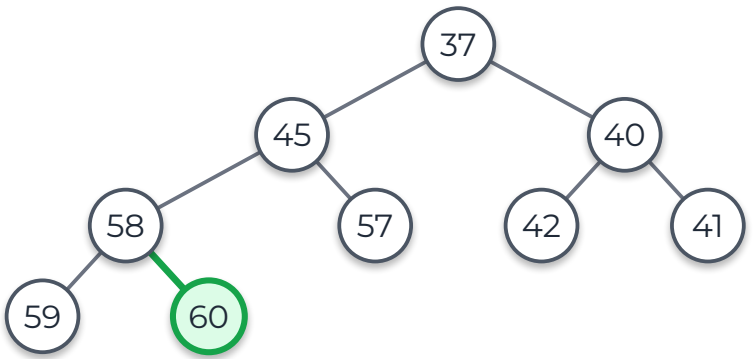
1. Tausch mit 37 (37 < 40)



2. Tausch mit 45 (45 < 57)



3. Tausch mit 58 (58 < 59) → Heap!



BuildHeap

Ziel

Aus einem **beliebigen** Binärbaum einen Heap machen.

Idee

- Blätter sind schon Heaps
- Alle Knoten **von hinten nach vorn** durchgehen (rechts → links, unterste Ebene → Wurzel)
- Für jeden Nicht-Blatt n: **DownHeap**
- Dann sind beide Teilbäume von n schon Heaps ✓

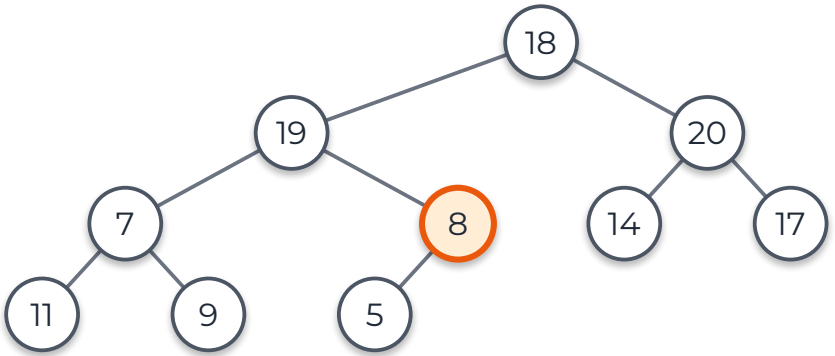
BuildHeap(B)

für jeden Knoten n, rückwärts in Ebenenreihenfolge:
    falls n kein Blatt:
        DownHeap(T(n), n)

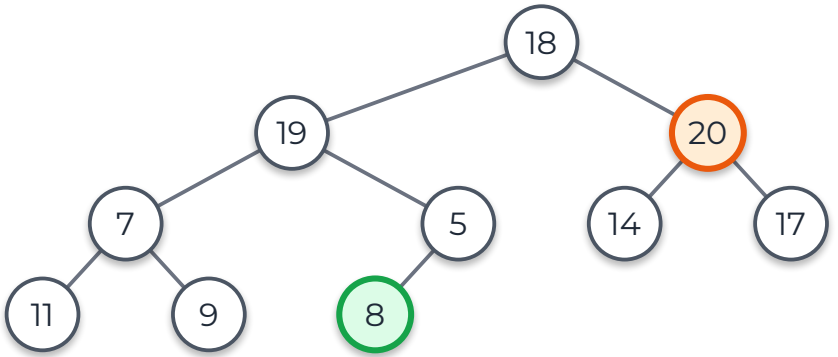
T(n) ist der Teilbaum von B mit Wurzel n.

BuildHeap: Beispiel

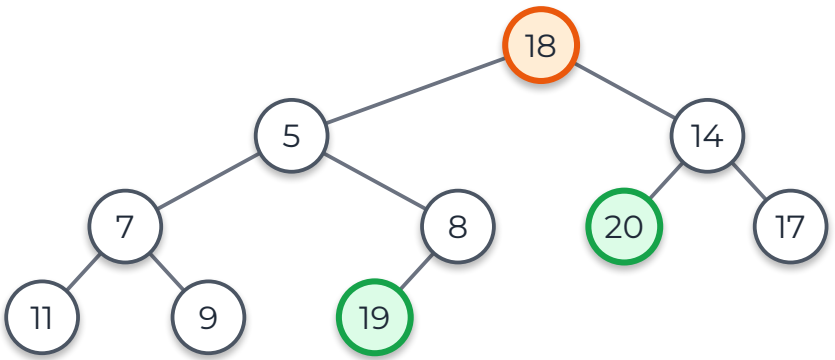
Start: Nicht-Blätter rückwärts 8, 7, 20, 19, 18



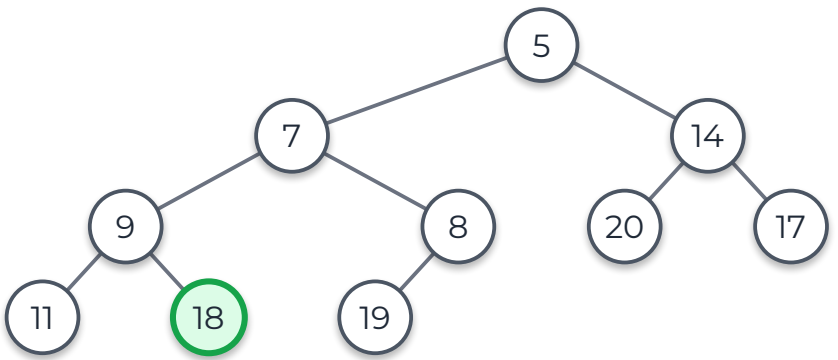
DownHeap(8): 8 ↔ 5 · DownHeap(7): –



DownHeap(20): 20 ↔ 14 · DownHeap(19): ↔ 5, 8



DownHeap(18): ↔ 5, 7, 9 → Heap!



Treap: Idee

- Binärer **Suchbaum** bzgl. des Schlüssels k – **und** Min-**Heap** bzgl. der Priorität p
- Jeder Knoten hat ein Paar (k, p)
- Die Priorität wird beim Einfügen **zufällig** gewählt
- search, insert, delete: **erwartet** O(log n)

Schlüssel k → Suchbaum-Bedingung: links kleiner, rechts größer

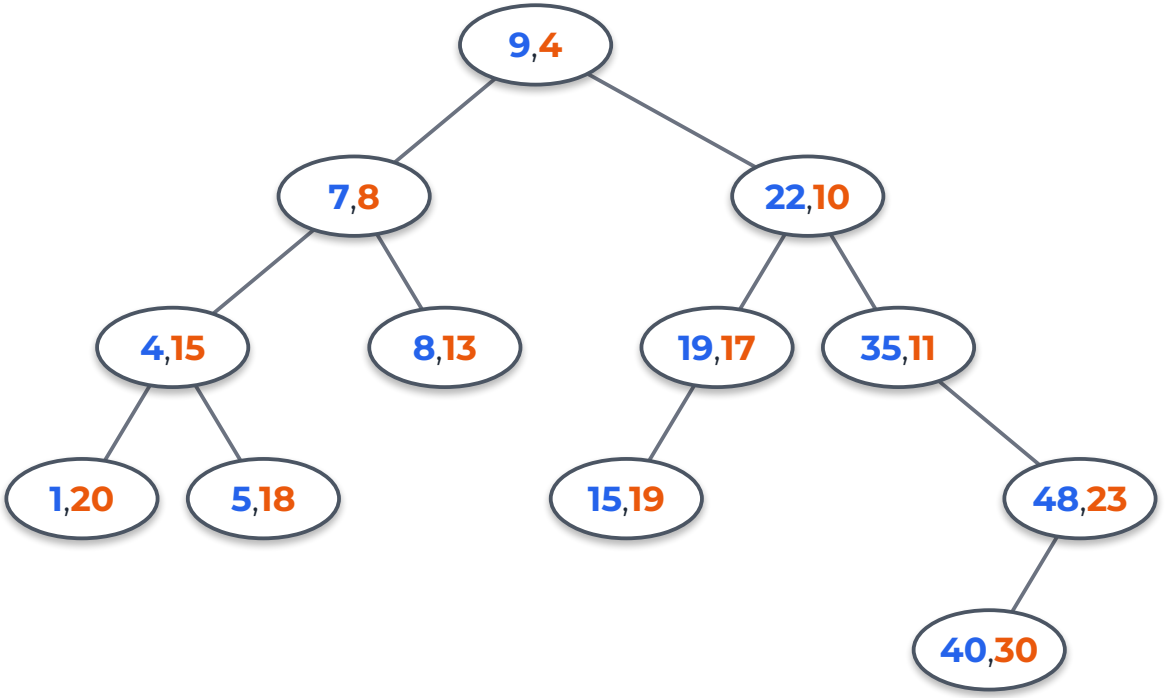
Priorität p → Min-Heap-Bedingung: Eltern ≤ Kinder

Treap: Beispiel

Blau: Schlüssel k

Orange: Priorität p

- Schlüssel: Suchbaum
- Priorität: Min-Heap
- Wurzel = kleinste Priorität



RotationHeap

- Solange p existiert und p größer als n ist, wird n nach oben rotiert:
  - n ist linkes Kind → **Rechtsrotation** um n
  - n ist rechtes Kind → **Linksrotation** um n
- Die Schlüsselreihenfolge bleibt erhalten → Suchbaum bleibt gültig



Treap: Suchen & Einfügen

Suchen von k

- Wie im binären Suchbaum – nur die Schlüssel vergleichen
- Keine Ausgleichsmaßnahmen nötig

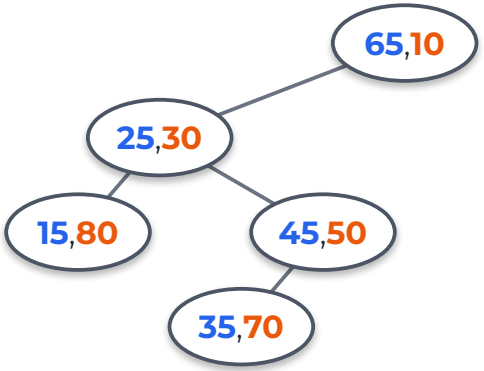
Einfügen von k

1. Zufällige Priorität p für k wählen
2. (k, p) wie im binären Suchbaum einfügen → neues **Blatt**, das evtl. die Heap-Bedingung stört
3. Mit **RotationHeap** das Blatt nach oben rotieren, bis die Heap-Bedingung gilt

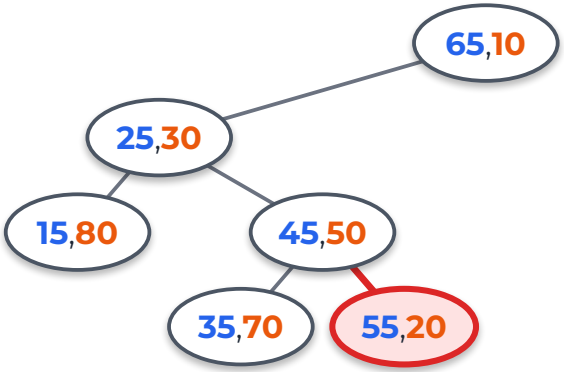
Rotationen ändern nur die Form, **nicht** die Reihenfolge der Schlüssel → die Suchbaum-Bedingung bleibt erhalten.

Einfügen: Beispiel (55, 20)

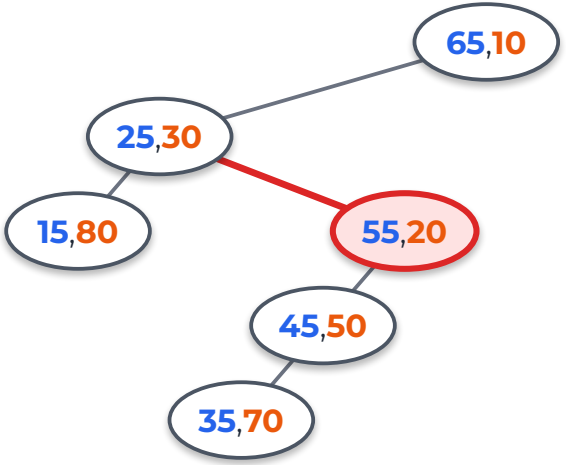
Start



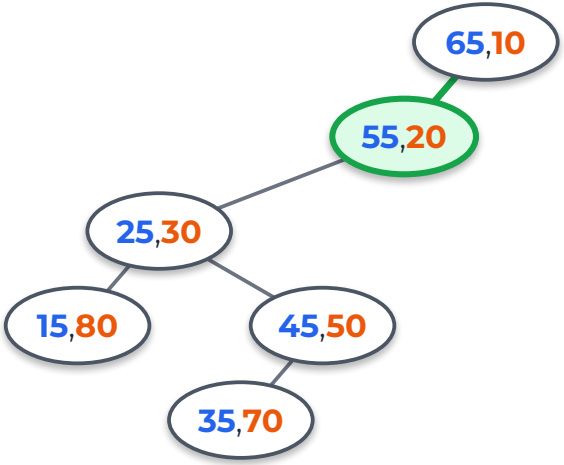
1. Einfügen wie im Suchbaum: neues Blatt (55,20)



2. Links-Rotation um (55,20): 20 < 50



3. Links-Rotation um (55,20): 20 < 30 → Treap!



Treap: Löschen

Genau **umgekehrt** zum Einfügen. Ist k nicht im Treap, ist nichts zu tun.

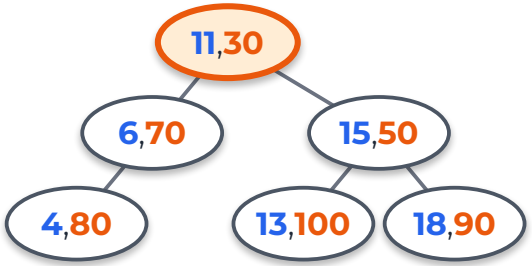
1. Knoten n mit Schlüssel k suchen
2. n per Rotation **nach unten** bewegen, bis n ein Blatt ist – immer um das Kind mit der **kleineren Priorität** rotieren
3. Blatt n entfernen

Warum funktioniert das?

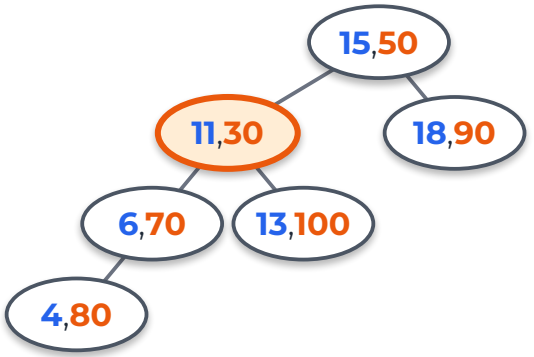
- Rotation erhält die Suchbaum-Eigenschaft
- Rotation um das Kind mit kleinerer Priorität erhält die Heap-Eigenschaft
- Ein Blatt kann ohne Probleme entfernt werden

Löschen: Beispiel (11, 30)

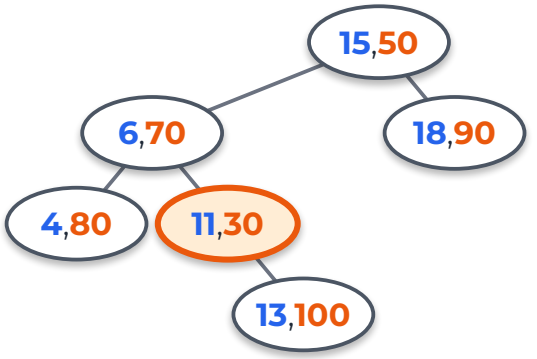
Start: lösche (11,30)



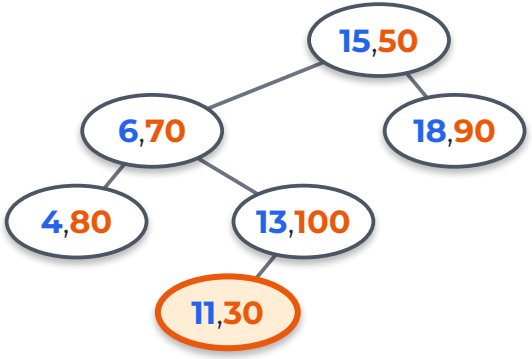
1. Links-Rot. um (15,50)



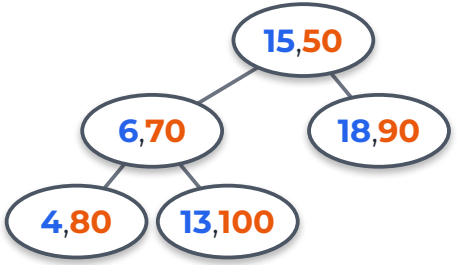
2. Rechts-Rot. um (6,70)



3. Links-Rot. um (13,100)



4. Blatt (11,30) entfernen



Immer zur kleineren Priorität:

(6,70) oder (15,50)? → 50

(6,70) oder (13,100)? → 70

nur (13,100) übrig → 100

Heap vs. Treap

	Heap	Treap
Struktur	Binärbaum mit Heap-Bedingung	Suchbaum (Schlüssel) + Min-Heap (Priorität)
Geordnet?	nur Eltern ≤ Kinder	Suchbaum bzgl. k
Reparatur	DownHeap, UpHeap, BuildHeap	RotationHeap
Einsatz	Priority Queue, Sortieren	geordnete Menge (Dictionary)
Laufzeit	DownHeap/UpHeap O(log n) *, BuildHeap O(n) *	search, insert, delete: erwartet O(log n)

* Ergänzung zu den Vorlesungsfolien